

Gadgets Gram Backend Engineering Blueprint

Phase 1: Define Business Flow First

Before writing a single line of code, map the customer journey.

```
Visitor
↓
Landing Page
↓
Order Form Submit
↓
Backend Validation
↓
Order Database
↓
Admin Panel
↓
Courier Booking
↓
Customer Notification
↓
Order Status Tracking
↓
Delivered / Returned
```

Backend exists to support this flow.

Phase 2: Core Backend Modules

Module 1: Authentication

Purpose:
Secure admin access.

Features:

- Admin Login
- Password Reset
- Session Management
- Role Management

Roles:

Super Admin
Admin
Customer Support
Marketing

Recommended:

JWT Authentication
Refresh Tokens
HTTP Only Cookies

Module 2: Order Management System

Most important module.

Database Table:

Orders

Fields:

id
customer_name
phone
alternative_phone
address
district
product_id
quantity
price
delivery_charge
subtotal
status
courier_tracking_id
notes
created_at
updated_at

Status Flow:

Pending
Confirmed
Packed
Courier Booked
In Transit
Delivered
Returned
Cancelled

Never hardcode statuses.

Use enums.

Module 3: Product Management

Database:

Products

Fields:

id
title
slug
description
price
sale_price
stock
images
status
created_at
updated_at

Features:

Create Product
Edit Product
Archive Product
Inventory Tracking

Module 4: Customer Management

Store:

Customer Data

Fields:

name
phone
email
orders_count
lifetime_value
last_order_date

Purpose:

Retargeting
Repeat Sales
Customer Service

Module 5: Analytics

Track:

Orders
Revenue
Conversion Rate
Traffic Sources
Best Selling Products
Return Rate

Dashboard Metrics:

Today's Orders
Weekly Orders
Monthly Revenue
ROAS

Module 6: Notification Service

Triggers:

Order Received
Order Confirmed
Courier Booked
Delivered

Channels:

SMS
WhatsApp
Email

Recommended:

Queue-based system.

Never send directly from API.

Module 7: Courier Integration

For Bangladesh:

Possible Integrations:

- Steadfast Courier
- Pathao Courier
- RedX

Flow:

```
Order Confirmed
↓
Create Consignment
↓
Store Tracking Number
↓
Sync Delivery Status
```

Phase 3: Database Architecture

Recommended:

PostgreSQL

Why:

Reliable
Scalable
Cheap
Excellent indexing

Tables:

```
users
roles
products
orders
customers
notifications
audit_logs
courier_shipments
settings
```

Phase 4: API Architecture

Structure:

/api/v1

Examples:

POST /orders

GET /orders

GET /orders/:id

PUT /orders/:id

DELETE /orders/:id

Products:

GET /products

POST /products

PUT /products/:id

Admin:

POST /auth/login

POST /auth/logout

Phase 5: Backend Tech Stack

For a modern SaaS-quality implementation:

Node.js
NestJS
PostgreSQL
Redis
Docker

Why NestJS?

Enterprise Structure
Scalable
TypeScript Native
Easy Dependency Injection
Clean Architecture

Folder Structure:

src/

auth/
orders/
products/
customers/
notifications/
courier/
analytics/

common/
config/
database/

Phase 6: Security Requirements

Mandatory:

Rate Limiting
CSRF Protection
JWT Validation
Input Sanitization
SQL Injection Protection
XSS Protection

Store:

Passwords = bcrypt

Never store plaintext passwords.

Phase 7: Background Jobs

Use Redis Queue.

Tasks:

SMS Sending
Email Sending
WhatsApp Sending
Courier Sync
Analytics Aggregation

Recommended:

BullMQ
Redis

Phase 8: Logging & Monitoring

Implement:

Request Logs
Error Logs
Audit Logs
Admin Activity Logs

Recommended:

Winston
Grafana
Prometheus

Phase 9: Deployment Architecture

Production Setup:

Cloud VPS
↓
Docker
↓
Nginx
↓
NestJS API
↓
PostgreSQL
↓
Redis

Example:

Ubuntu 24.04
Docker
Docker Compose
Nginx Reverse Proxy
SSL

Phase 10: CI/CD Pipeline

Flow:

GitHub Push
↓
GitHub Actions

↓
Build
↓
Test
↓
Deploy
↓
Restart Containers

Never deploy manually once live.

Phase 11: Production Checklist

Before launch:

Performance

- Gzip Enabled
- Image CDN
- Redis Cache

Security

- SSL
- Firewall
- Fail2Ban
- Secrets Management

Database

- Daily Backup
- Weekly Backup Verification

Monitoring

- Uptime Monitoring
 - Error Alerts
-

Final Architecture

Frontend (Landing Page)

↓ API

NestJS Backend

↓

PostgreSQL

↓

Redis Queue

↓

Notification Service

↓

Courier Integration

↓

Admin Dashboard

For **Gadgets Gram**, I would avoid WordPress/PHP if you're building something meant to scale. A **NestJS + PostgreSQL + Redis + Docker** stack is the sweet spot: inexpensive, maintainable, and capable of handling thousands of orders per day while remaining manageable by AI coding agents.